

REPORT

ECG SIGNAL PROCESSING AND ARRHYTHMIA DETECTION USING MACHINE LEARNING

SUBMITTED BY:

S.No.	Name
1	Sarthak Gupta

ABSTRACT

This project presents an automated system for ECG signal processing and arrhythmia detection using machine learning techniques. Electrocardiogram (ECG) signals are widely used for diagnosing heart-related abnormalities, but manual analysis is time-consuming and prone to error.

In this work, ECG signals from the MIT-BIH Arrhythmia Database are processed using digital filtering techniques to remove noise such as baseline wander and power line interference. R-peak detection is performed to identify heartbeats and calculate RR intervals.

Relevant features are extracted from the processed signals and used as input to machine learning models such as Support Vector Machine (SVM) and Random Forest. These models classify ECG signals into normal and abnormal categories.

The system is evaluated using performance metrics such as accuracy, precision, and recall. The results demonstrate that machine learning can effectively automate ECG analysis and improve diagnostic efficiency.

This project provides a scalable and efficient framework for intelligent healthcare systems and can be extended for real-time monitoring applications.

TABLE OF CONTENTS

CHAPTER NO.	TOPIC	PAGE NO.
	ABSTRACT	4
1	INTRODUCTION 1.1 INTRODUCTION 1.2 OBJECTIVES 1.3 SCOPE OF THE PROJECT	10 – 12 10 11 12
2	LITERATURE SURVEY 2.1 INTRODUCTION TO LITERATURE SURVEY 2.2 REVIEW OF EXISTING WORK 2.3 COMPARATIVE ANALYSIS OF TECHNIQUES 2.4 IDENTIFIED RESEARCH GAPS 2.5 PROPOSED APPROACH IN THIS PROJECT 2.6 SUMMARY	13-17 13 14 15 16 17
3	SYSTEM DESIGN AND METHODOLOGY 3.1 INTRODUCTION 3.2 SYSTEM BLOCK DIAGRAM 3.3 BLOCK DIAGRAM EXPLANATION 3.4 WORKING PRINCIPLE 3.5 METHODOLOGY 3.6 FLOWCHART 3.7 ALGORITHM 3.8 MATHEMATICAL MODELING OF ECG SIGNAL 3.9 SIGNAL FILTERING TECHNIQUES 3.10 FEATURE ENGINEERING 3.11 DESIGN CONSIDERATIONS 3.12 LIMITATIONS 3.13 SUMMARY	18 – 29 18 19 20 – 21 22 23 24 25 26 27 28 29

4	IMPLEMENTATION 4.1 INTRODUCTION 4.2 TOOLS AND TECHNOLOGIES USED 4.3 DATASET DESCRIPTION 4.4 DATA PREPROCESSING 4.5 R – PEAK DETECTION 4.6 FEATURE EXTRACTION 4.7 FEATURE SELECTION AND SCALING 4.8 MACHINE LEARNING MODELS 4.9 MODEL TRAINING AND TESTING 4.10 PERFORMANCE EVALUATION 4.11 CONFUSION MATRIX 4.12 VISUALIZATION OF RESULTS 4.13 IMPLEMENTATION CHALLENGES 4.14 TRAIN – TEST SPLIT AND VALIDATION 4.15 HYPERPARAMETER TUNING 4.16 OVERFITTING AND UNDERFITTING 4.17 COMPUTATIONAL COMPLEXITY 4.18 SYSTEM PERFORMANCE OPTIMIZATION 4.19 IMPLEMENTATION ENVIRONMENT 4.20 SUMMARY	30 – 40 30 31 32 33 34 35 36 36 – 37 37 38 39 40
5	RESULT AND DISCUSSION 5.1 INTRODUCTION 5.2 PREPROCESSING RESULTS 5.3 R – PEAK DETECTION RESULTS 5.4 FEATURE EXTRACTION RESULTS 5.5 MODEL PERFORMANCE EVALUATION 5.6 CONFUSION MATRIX ANALYSIS 5.7 CLASSIFICATION REPORT ANALYSIS	41 – 49 41 41 – 42 43 44 45 46 47

	5.8 COMPARATIVE ANALYSIS	47 – 48
	5.9 OBSERVATIONS	49
	5.10 SUMMARY	
	CONCLUSION	50
	APPLICATIONS	51
	FUTURE SCOPE	51
	REFERENCES	52
	APPENDIX A – SOURCE CODE	53 - 55

LIST OF FIGURES

FIGURE NO.	FIGURE TITLE	PAGE NO.
Fig. 3.1	BLOCK DIAGRAM OF ECG SIGNAL PROCESSING	19
Fig. 3.2	FLOWCHART OF ECG SIGNAL PROCESSING	24
Fig. 4.1	DIFFERENT TYPES OF ECG SIGNALS	36
Fig. 5.1	RAW ECG SIGNAL	42
Fig. 5.2	FILTERED ECG SIGNAL	42
Fig. 5.3	R – PEAK DETECTION	43
Fig. 5.4	FEATURE EXTRACTION	44
Fig. 5.5	TRAIN SIZE AND TEST SIZE	45
Fig. 5.6	SUPPORT VECTOR MACHINE AND RANDOM FOREST ACCURACY	45
Fig. 5.7	CONFUSION MATRIX ANALYSIS	46
Fig. 5.8	CLASSIFICATION REPORT ANALYSIS	47
Fig. 5.9	MODEL ACCURACY COMPARISON (SVM VS RF)	48
Fig. 5.10	CLASS DISTRIBUTION	48

CHAPTER – 1

INTRODUCTION

1.1 INTRODUCTION

Cardiovascular diseases (CVDs) are one of the leading causes of death globally, accounting for millions of deaths every year. According to global health reports, early detection and continuous monitoring of heart conditions play a crucial role in reducing mortality rates. Among various diagnostic tools available, the Electrocardiogram (ECG) is widely used for recording the electrical activity of the heart and identifying abnormalities in cardiac rhythm.

An ECG signal represents the depolarization and repolarization of the heart muscles, which are essential for maintaining proper blood circulation. The ECG waveform consists of several key components including the P-wave, QRS complex, and T-wave. Each of these components corresponds to a specific phase of cardiac activity. For instance, the P-wave represents atrial depolarization, the QRS complex indicates ventricular depolarization, and the T-wave corresponds to ventricular repolarization. Any deviation in these waveforms can indicate potential cardiac abnormalities such as arrhythmias.

Arrhythmia refers to an irregular heartbeat condition where the heart may beat too fast (tachycardia), too slow (bradycardia), or in an irregular pattern. Detecting arrhythmias at an early stage is extremely important as they can lead to severe complications including stroke, heart failure, or even sudden cardiac arrest. However, manual interpretation of ECG signals requires expert knowledge and can be time-consuming, especially when dealing with long-duration recordings such as 24-hour Holter monitoring.

In addition to this, ECG signals are often affected by various types of noise and interference, such as baseline wander, muscle artifacts, electrode motion artifacts, and power line interference (50 Hz). These noise components can distort the signal and make accurate diagnosis more difficult. Therefore, preprocessing of ECG signals is a critical step before performing any analysis.

With the rapid advancement in technology, machine learning techniques have emerged as powerful tools for analyzing biomedical signals. Machine learning algorithms can automatically learn patterns from data and classify signals with high accuracy. When combined with digital signal processing techniques, they provide an efficient solution for automated ECG analysis.

1.2 OBJECTIVES

The primary objective of this project is to design and implement an automated ECG signal processing system capable of detecting arrhythmia using machine learning techniques. The project aims to combine the principles of digital signal processing with modern machine learning algorithms to achieve accurate classification of ECG signals.

The detailed objectives of the project are as follows:

- To study the fundamental concepts of ECG signals and understand the physiological significance of different waveform components such as P-wave, QRS complex, and T-wave.
- To acquire ECG data from a standard and reliable dataset such as the MIT-BIH Arrhythmia Database, which provides annotated signals for research purposes.
- To preprocess the raw ECG signals by removing noise components such as baseline drift, high-frequency noise, and power line interference using digital filtering techniques.
- To implement R-peak detection algorithms to accurately identify heartbeats and calculate RR intervals, which are crucial for heart rate variability analysis.
- To extract meaningful features from ECG signals, including time-domain features (RR interval, heart rate), statistical features (mean, variance), and frequency-domain features using techniques such as Fast Fourier Transform (FFT).
- To implement supervised machine learning algorithms such as Support Vector Machine (SVM) and Random Forest for classification of ECG signals into normal and abnormal categories.
- To evaluate the performance of the classification model using standard metrics such as accuracy, precision, recall, F1-score, and confusion matrix.
- To analyze the results and compare the performance of different machine learning models to identify the most effective approach.
- To develop a scalable system that can be extended for real-time monitoring and integration with wearable devices in future.

1.3 SCOPE OF THE PROJECT

The scope of this project is extensive and has both national and global relevance due to the increasing prevalence of cardiovascular diseases worldwide. The proposed system focuses on developing an automated ECG signal processing and arrhythmia detection framework that can be applied in various healthcare environments.

At a global level, the system can be integrated into modern healthcare infrastructures, including hospitals, diagnostic laboratories, and telemedicine platforms. With the increasing adoption of digital health technologies, automated ECG analysis systems can significantly improve diagnostic efficiency and reduce the workload on medical professionals.

The system can also be incorporated into wearable health monitoring devices such as smartwatches and portable ECG monitors. These devices continuously monitor heart activity and generate large volumes of data, which can be effectively analyzed using machine learning algorithms. By integrating this project with wearable technology, real-time detection of arrhythmias becomes possible, enabling early intervention and preventive healthcare.

From a national perspective, the project aligns with initiatives aimed at improving healthcare accessibility and digital transformation. In countries like India, where there is a shortage of specialized healthcare professionals in rural and semi-urban areas, automated ECG analysis systems can play a crucial role in providing basic diagnostic support.

The system can also be deployed in remote healthcare centers where access to advanced medical facilities is limited. By integrating this system with cloud-based platforms, ECG data can be transmitted to specialists for further analysis, thereby improving the overall quality of healthcare services.

In addition to healthcare applications, this project also contributes to academic research in the field of biomedical signal processing and machine learning. It provides a foundation for further research and development in areas such as deep learning-based ECG analysis, real-time monitoring systems, and predictive healthcare analytics.

Furthermore, the system can be extended to support multi-class classification of different types of arrhythmias, rather than simple binary classification. Advanced techniques such as convolutional neural networks (CNN) and recurrent neural networks (RNN) can be implemented to improve classification accuracy.

CHAPTER – 2

LITERATURE SURVEY

2.1 INTRODUCTION TO LITERATURE SURVEY

The rapid growth in cardiovascular diseases has increased the demand for automated systems capable of analyzing ECG signals efficiently. Over the past few decades, significant research has been carried out in the field of ECG signal processing and arrhythmia detection using both traditional signal processing techniques and modern machine learning approaches.

The literature survey provides a detailed understanding of existing methods, technologies, and approaches used by researchers in this domain. It helps in identifying the strengths and limitations of previous work and provides a foundation for developing improved systems.

Early research in ECG analysis primarily focused on signal processing techniques such as filtering and peak detection. However, with the advancement of computational capabilities and availability of large datasets, machine learning and deep learning techniques have become widely used for automated ECG classification.

This chapter presents a detailed review of significant research contributions in the field of ECG signal processing and arrhythmia detection.

2.2 REVIEW OF EXISTING WORK

One of the most fundamental contributions in ECG signal processing was made by Pan and Tompkins in 1985, who developed a real-time QRS detection algorithm. Their method utilized digital filtering, differentiation, squaring, and adaptive thresholding to accurately detect R-peaks in ECG signals. This algorithm became a benchmark in ECG analysis and is still widely used in many applications.

Later, Moody and Mark introduced the MIT-BIH Arrhythmia Database, which became a standard dataset for evaluating ECG classification algorithms. The availability of annotated ECG signals enabled researchers to develop and test automated classification systems with improved accuracy.

In 2004, De Chazal and colleagues proposed an automatic heartbeat classification system using ECG morphology and RR interval features. Their approach combined time-domain features with machine learning classifiers to achieve reliable classification results. However, the performance of the system depended heavily on manual feature extraction, which limited its scalability.

Another significant contribution was made by Osowski et al., who applied Support Vector Machine (SVM) for ECG classification. Their study demonstrated that SVM can effectively classify ECG signals with high accuracy due to its ability to handle high-dimensional data. However, the model required careful selection of kernel functions and parameters to achieve optimal performance.

With the advancement of machine learning, ensemble methods such as Random Forest have also been used for ECG classification. Random Forest combines multiple decision trees to improve accuracy and reduce overfitting. It has been widely adopted due to its robustness and ability to handle noisy data.

In recent years, deep learning techniques have gained significant attention in ECG analysis. Acharya et al. (2017) proposed a deep convolutional neural network (CNN) model for automated ECG classification. Their approach eliminated the need for manual feature extraction by directly learning features from raw ECG signals.

Similarly, Yıldırım et al. (2019) introduced a hybrid CNN-LSTM model that combines spatial and temporal feature extraction. The CNN component extracts spatial features, while the LSTM captures temporal dependencies in ECG signals. This approach achieved high classification accuracy but required high computational resources.

2.3 COMPARATIVE ANALYSIS OF TECHNIQUES

The analysis of existing research reveals that ECG classification techniques can be broadly categorized into three main approaches:

1. Traditional Signal Processing Methods
2. Machine Learning-Based Methods
3. Deep Learning-Based Methods

Traditional signal processing methods focus on filtering, peak detection, and rule-based classification. These methods are simple and computationally efficient but lack adaptability to complex patterns.

Machine learning-based methods, such as SVM and Random Forest, provide better classification accuracy by learning patterns from data. They require feature extraction but offer a good balance between accuracy and computational efficiency.

Deep learning-based methods, such as CNN and LSTM, provide the highest accuracy by automatically learning features from raw data. However, they require large datasets and high computational power, making them less suitable for low-resource environments.

Based on this comparison, machine learning methods are considered suitable for this project as they provide reliable performance while maintaining computational efficiency.

2.4 IDENTIFIED RESEARCH GAPS

Despite significant advancements in ECG signal processing, several challenges still exist:

- Many systems rely heavily on manual feature extraction, which limits scalability.
- Deep learning models require large datasets and high computational resources.
- Noise in ECG signals can still affect classification accuracy.
- Real-time implementation remains a challenge for complex models.
- Many existing systems focus only on accuracy without considering computational efficiency.

These gaps highlight the need for a system that balances accuracy, efficiency, and scalability.

2.5 PROPOSED APPROACH IN THIS PROJECT

To address the identified gaps, this project proposes a hybrid approach that combines digital signal processing techniques with machine learning algorithms.

The system uses preprocessing techniques to remove noise and improve signal quality. Feature extraction is performed to obtain meaningful parameters from ECG signals. Machine learning algorithms such as SVM and Random Forest are used for classification.

This approach provides a balance between accuracy and computational efficiency, making it suitable for practical applications.

2.6 SUMMARY

This chapter presented a detailed review of existing research in ECG signal processing and arrhythmia detection. It highlighted the evolution of techniques from traditional signal processing to advanced machine learning and deep learning methods.

The analysis of literature helped in identifying key challenges and research gaps, which are addressed in the proposed system. The next chapter presents the system design and architecture of the proposed project.

CHAPTER – 3

SYSTEM DESIGN AND METHODOLOGY

3.1 INTRODUCITON

This chapter describes the overall design and methodology of the proposed system for ECG signal processing and arrhythmia detection using machine learning. The system is designed in a modular and systematic manner to ensure efficient processing of ECG signals and accurate classification of cardiac conditions.

The design includes multiple stages such as data acquisition, signal preprocessing, feature extraction, and classification. Each stage plays a crucial role in transforming raw ECG signals into meaningful information that can be used for diagnosis.

The system follows a pipeline-based architecture, where the output of one stage serves as the input for the next stage. This structured approach ensures clarity, scalability, and ease of implementation.

3.2 SYSTEM BLOCK DIAGRAM

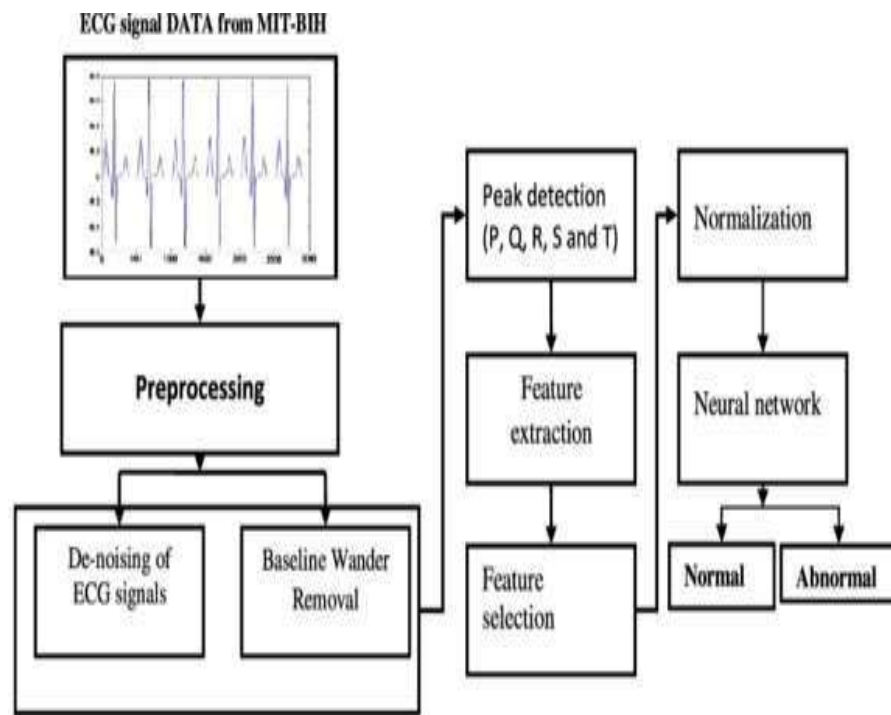


Fig 3.1 BLOCK DIAGRAM OF ECG SIGNAL PROCESSING AND ARRHYTHMIA DETECTION USING MACHINE LEARNING

3.3 **BLOCK DIAGRAM EXPLANATION**

The system consists of the following major components:

1. ECG Data Acquisition

The ECG signals are obtained from the MIT-BIH Arrhythmia Database, which is a standard dataset widely used in biomedical research. The dataset contains annotated ECG recordings sampled at 360 Hz.

2. Signal Preprocessing

Raw ECG signals contain various types of noise such as baseline wander, muscle artifacts, and power line interference.

To remove these unwanted components, digital filtering techniques are applied:

- Bandpass filter (0.5–40 Hz)
- Signal normalization

This stage improves signal quality and prepares it for further analysis.

3. R-Peak Detection

R-peaks are the most prominent points in the ECG signal and are used to identify heartbeats.

Peak detection algorithms are used to detect R-peaks and calculate RR intervals, which represent the time difference between consecutive heartbeats.

4. Feature Extraction

After detecting R-peaks, meaningful features are extracted from the ECG signal. These include:

- Time-domain features (RR interval, heart rate)
- Statistical features (mean, variance)
- Morphological features (QRS width, amplitude)

These features represent the characteristics of the ECG signal.

5. Feature Selection and Scaling

Not all extracted features are useful for classification. Therefore, important features are selected, and scaling is applied to normalize the data.

This improves model performance and reduces computational complexity.

6. Machine Learning Classifier

The processed features are fed into machine learning models such as:

- Support Vector Machine (SVM)
- Random Forest

These models classify the ECG signal into normal or arrhythmic categories.

7. Output

The final output of the system is the classification result:

- Normal
- Arrhythmia

3.4 WORKING PRINCIPLE

The working principle of the system is based on the integration of digital signal processing and machine learning techniques.

Initially, ECG signals are collected from the dataset and passed through preprocessing filters to remove noise. The cleaned signal is then analyzed to detect R-peaks, which helps in identifying heartbeats.

Once the peaks are detected, various features are extracted to represent the signal in a structured form. These features are then used as input to machine learning algorithms, which classify the signal based on learned patterns.

The entire process is automated and operates in a sequential manner, ensuring efficient and accurate analysis of ECG signals.

3.5 METHODOLOGY

The methodology of the proposed system is divided into the following steps:

1. Data Acquisition

ECG data is collected from the MIT-BIH dataset.

2. Preprocessing

Noise removal is performed using digital filters.

3. R-Peak Detection

Heartbeats are identified using peak detection algorithms.

4. Feature Extraction

Important features are extracted from ECG signals.

5. Model Training

Machine learning models are trained using extracted features.

6. Testing and Validation

The model is tested using unseen data.

7. Performance Evaluation

Accuracy, precision, and recall are calculated.

This structured methodology ensures reliable system performance.

3.6 FLOWCHART

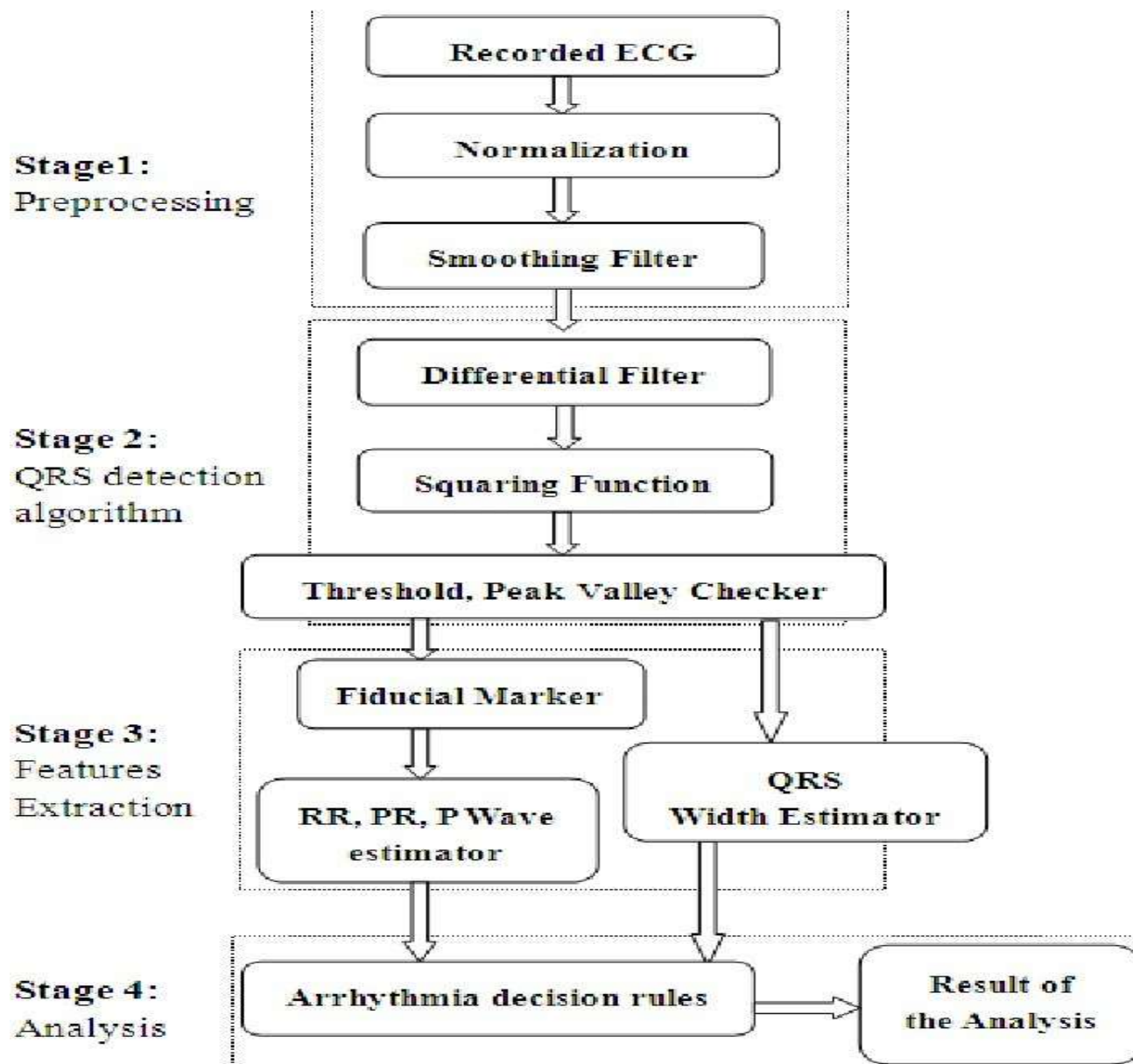


Fig 3.2 FLOWCHART OF ECG SIGNAL PROCESSING AND ARRHYTHMIA DETECTION USING MACHINE LEARNING

3.7 ALGORITHM

The algorithm of the system is as follows:

- Start the system
- Load ECG dataset
- Apply preprocessing filters
- Detect R-peaks
- Extract features
- Split dataset into training and testing
- Train machine learning model
- Test the model
- Evaluate performance
- Display output
- Stop

3.8 MATHEMATICAL MODELING OF ECG SIGNAL

Electrocardiogram signals can be mathematically represented as a time-varying signal composed of multiple waveform components. The ECG signal $x(t)$ can be expressed as:

$$x(t) = P(t) + QRS(t) + T(t) + n(t)$$

Where:

- $P(t)$ = P – wave component
- $QRS(t)$ = QRS complex
- $T(t)$ = T – wave
- $n(t)$ = Noise component

The RR interval is calculated as:

$$RR = t_{R(i+1)} - t_{R(i)}$$

Heart Rate (HR) is given by:

$$HR = \frac{60}{RR}$$

This mathematical representation helps in analyzing ECG signals quantitatively and extracting meaningful features for classification.

3.9 SIGNAL FILTERING TECHNIQUES

ECG signals are often contaminated with noise, which can affect analysis accuracy. Therefore, filtering techniques are applied:

- Bandpass Filter

Removes low-frequency baseline drift and high-frequency noise.
Typical range: **0.5 Hz to 40 Hz**

- Notch Filter

Removes power line interference (50 Hz).

- Normalization

Scales signal amplitude for uniform processing.

These techniques improve signal quality and enhance feature extraction accuracy.

3.10 FEATURE ENGINEERING

Feature engineering plays a crucial role in machine learning performance.

Extracted Features:

Time Domain:

- RR interval
- Heart rate
- Mean RR

Statistical Features:

- Mean
- Variance
- Standard deviation

Morphological Features:

- QRS width
- Peak amplitude

Feature selection helps remove redundant data and improve classification efficiency.

3.11 DESIGN CONSIDERATIONS

While designing the system, the following factors were considered:

- Accuracy of classification
- Noise reduction efficiency
- Computational complexity
- Scalability of system
- Real-time implementation feasibility

These considerations ensure that the system is both practical and efficient.

3.12 LIMITATIONS OF DESIGN

Even though the system performs effectively, some limitations exist:

- Performance depends on quality of input data
- Noise may affect peak detection accuracy
- Machine learning models require proper tuning
- Real-time implementation not included

3.13 SUMMARY

This chapter presented the system design and methodology of the proposed ECG signal processing system. The block diagram and flowchart provided a clear understanding of the workflow.

CHAPTER – 4

IMPLEMENTATION

4.1 INTRODUCTION

This chapter presents the practical implementation of the proposed ECG signal processing and arrhythmia detection system. The implementation focuses on processing real ECG data, extracting meaningful features, and applying machine learning algorithms for classification.

The system is developed using Python programming language due to its simplicity, flexibility, and availability of powerful libraries for signal processing and machine learning. The implementation includes data acquisition, preprocessing, feature extraction, model training, and performance evaluation.

4.2 TOOLS AND TECHNOLOGIES USED

The following tools and technologies are used in this project:

- Python Programming Language

Python is used as the primary programming language due to its ease of use and strong support for scientific computing.

- NumPy

Used for numerical computations and handling ECG signal arrays.

- SciPy

Used for signal processing operations such as filtering and frequency analysis.

- WFDB Library

Used for loading ECG signals from the MIT-BIH Arrhythmia Database.

- Matplotlib

Used for plotting ECG signals and visualization of results.

- Seaborn

Used for advanced visualization such as confusion matrix heatmaps.

- Scikit-learn

Used for implementing machine learning models such as SVM and Random Forest.

- TensorFlow/Keras (Optional)

Used for deep learning-based models if required.

4.3 DATASET DESCRIPTION

The dataset used in this project is the MIT-BIH Arrhythmia Database. It is one of the most widely used datasets for ECG analysis.

The dataset consists of multiple ECG recordings with annotations for each heartbeat. The signals are sampled at a frequency of 360 Hz, which provides sufficient resolution for detecting waveform components.

Each ECG record contains information about normal and abnormal heartbeats, which makes it suitable for supervised learning. The dataset is divided into training and testing sets to evaluate model performance.

4.4 DATA PREPROCESSING

Raw ECG signals contain noise and artifacts that can affect analysis accuracy. Therefore, preprocessing is an essential step.

- Noise Removal

Bandpass filtering is applied to remove baseline wander and high-frequency noise.

- Signal Normalization

The amplitude of ECG signals is normalized to ensure uniform scaling.

- Segmentation

The ECG signal is divided into segments based on R-peaks to analyze individual heartbeats.

Preprocessing improves signal quality and prepares it for feature extraction.

4.5 R – PEAK DETECTION

R-peak detection is a crucial step in ECG analysis. It helps in identifying heartbeats and calculating RR intervals.

Peak detection algorithms are used to locate the maximum points in the ECG signal corresponding to the QRS complex.

Once the R-peaks are detected, the RR interval is calculated as the time difference between consecutive peaks.

This information is used to analyze heart rate variability and detect abnormalities.

4.6 FEATURE EXTRACTION

Feature extraction is performed to convert ECG signals into a structured format suitable for machine learning.

Extracted Features:

- RR Interval
- Heart Rate
- Mean and Standard Deviation
- Signal Amplitude
- QRS Width

These features capture important characteristics of ECG signals and help in classification.

4.7 FEATURE SELECTION AND SCALING

Not all features contribute equally to classification. Therefore, feature selection is performed to remove irrelevant features.

Feature scaling is applied to normalize data and improve model performance.

This step reduces computational complexity and enhances accuracy.

4.8 MACHINE LEARNING MODELS

Two machine learning models are implemented in this project:

- Support Vector Machine (SVM)

SVM is a supervised learning algorithm used for classification. It works by finding an optimal hyperplane that separates data into different classes.

SVM is effective for binary classification and works well with high-dimensional data.

- Random Forest

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve accuracy and reduce overfitting.

It is robust and performs well with noisy data.

4.9 MODEL TRAINING AND TESTING

The dataset is divided into training and testing sets using an 80–20 split.

The training set is used to train the machine learning models, while the testing set is used to evaluate performance.

Cross-validation techniques can also be applied to improve reliability.

4.10 PERFORMANCE EVALUATION

The performance of the model is evaluated using the following metrics:

- Accuracy
- Precision
- Recall
- F1-score

These metrics provide a comprehensive evaluation of classification performance.

4.11 CONFUSION MATRIX

The confusion matrix is used to analyze classification results.

It shows:

- True Positive
- False Positive
- True Negative
- False Negative

This helps in understanding model performance in detail.

4.12 VISUALIZATION OF RESULTS

Visualization is used to understand ECG signals and model performance.

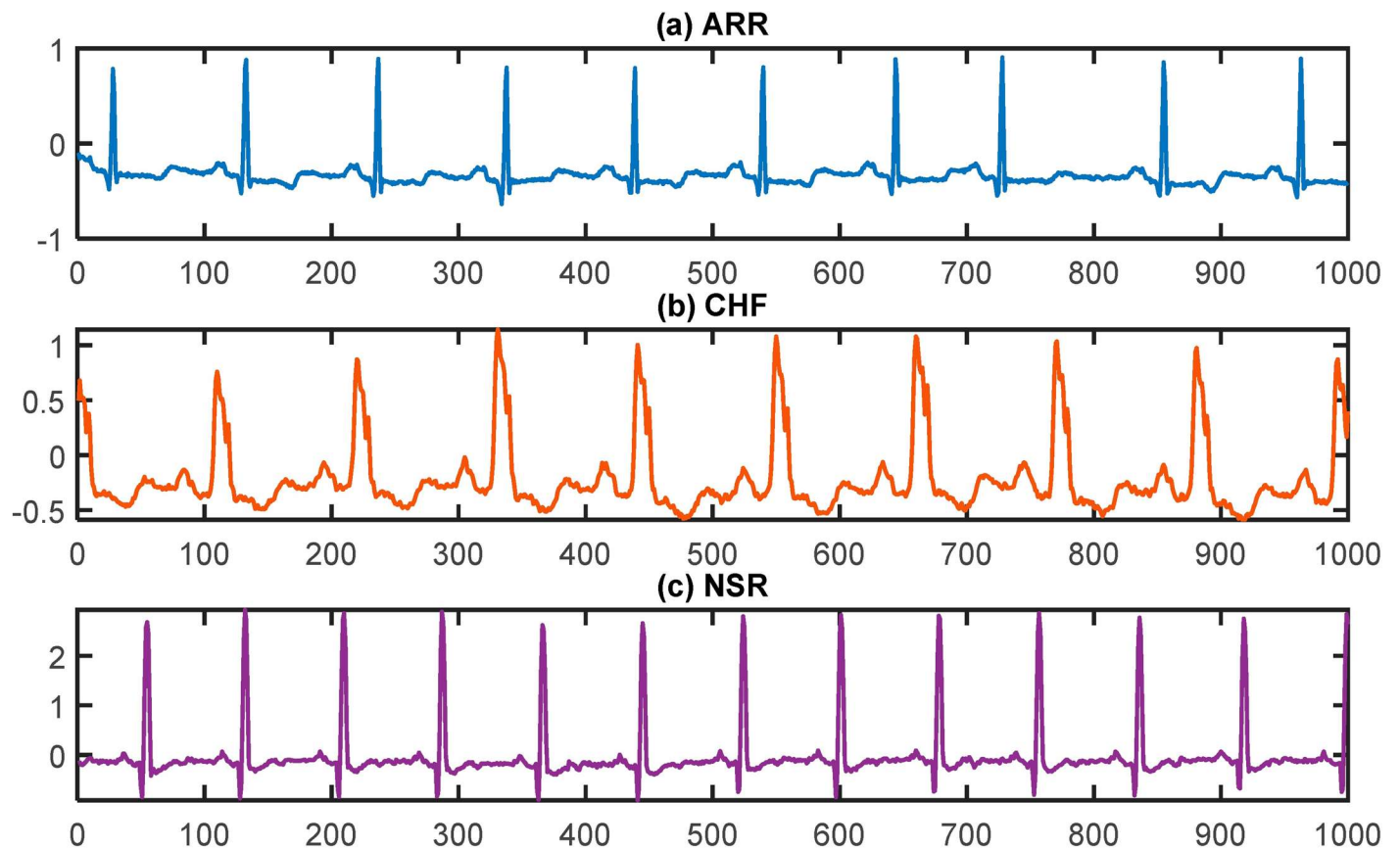


Fig 4.1 DIFFERENT TYPES OF ECG SIGNALS

The visualization of ECG signals for ARR, NSR, and CHF is presented in Figure 4.1. In Figure 4.1, it can be observed that all three ECG patterns have their own characteristic curve shapes. The ARR pattern exhibits irregular and abnormal waveforms, indicating an irregular heart rhythm. In contrast, the NSR pattern shows a regular and consistent waveform, indicating a healthy heart rhythm. Lastly, the CHF pattern displays a distorted waveform with varying amplitudes, representing the compromised function of the heart due to fluid accumulation. These distinct curves enable medical professionals to accurately diagnose and differentiate between different cardiac conditions by analyzing the ECG signals. By analyzing the irregular and abnormal waveforms, medical professionals can identify conditions such as atrial fibrillation or ventricular tachycardia, which require specific treatment approaches. On the other hand, the regular and consistent waveform of NSR helps confirm a healthy heart rhythm and rule out any significant cardiac abnormalities. The distorted waveform and varying amplitudes of the CHF pattern provide valuable insights into the compromised function of the heart, allowing doctors to prescribe appropriate medications and interventions to manage fluid accumulation and improve heart function. Overall, the analysis of ECG signals plays a crucial role in accurate diagnosis and effective treatment planning for patients with cardiac conditions.

4.13 IMPLEMENTATION CHALLENGES

During implementation, several challenges were encountered:

- Noise in ECG signals
- Difficulty in peak detection
- Feature selection complexity
- Model tuning

These challenges were addressed through preprocessing and optimization techniques.

4.14 TRAIN – TEST SPLIT AND VALIDATION

The dataset is divided into training and testing sets to evaluate model performance. Typically, an 80–20 split is used, where 80% of the data is used for training and 20% for testing.

Cross-validation techniques such as K-Fold Cross Validation can also be used to ensure that the model performs consistently across different subsets of data. This helps in reducing bias and improving generalization.

4.15 HYPERPARAMETER TUNING

Machine learning models require proper tuning of parameters to achieve optimal performance.

For SVM:

- Kernel type (linear, RBF)
- Regularization parameter (C)

For Random Forest:

- Number of trees
- Maximum depth
- Minimum samples per leaf

Hyperparameter tuning improves model accuracy and prevents overfitting.

4.16 OVERFITTING AND UNDERFITTING

Overfitting occurs when the model performs well on training data but poorly on testing data. Underfitting occurs when the model fails to capture patterns in the data.

To avoid these issues:

- Use cross-validation
- Use proper feature selection
- Apply regularization techniques

4.17 COMPUTATIONAL COMPLEXITY

The performance of machine learning models also depends on computational efficiency.

SVM:

- High accuracy but computationally intensive

Random Forest:

- Faster training but requires more memory

This project balances accuracy and efficiency by using optimized models.

4.18 SYSTEM PERFORMANCE OPTIMIZATION

Several techniques are used to improve system performance:

- Feature scaling
- Removing redundant features
- Efficient data splitting
- Proper model selection

These steps ensure reliable and efficient classification.

4.19 IMPLEMENTATION ENVIRONMENT

The system is implemented in a standard computing environment with:

- Python installed
- Required libraries (NumPy, SciPy, Scikit-learn)
- Jupyter Notebook / VS Code

The implementation is platform-independent and can run on any system with Python support.

4.20 SUMMARY

This chapter presented the implementation details of the proposed system. It covered tools, dataset, preprocessing, feature extraction, machine learning models, and evaluation metrics.

CHAPTER – 5

RESULT AND DISCUSSION

5.1 INTRODUCITON

This section presents the results obtained from the implementation of the ECG signal processing and arrhythmia detection system using machine learning techniques. The system was tested on ECG signals obtained from the MIT-BIH Arrhythmia Database.

The results demonstrate the effectiveness of preprocessing, feature extraction, and classification techniques used in the project. The performance of machine learning models such as Support Vector Machine (SVM) and Random Forest was evaluated using standard metrics.

5.2 PREPROCESSING RESULTS

The preprocessing stage plays a crucial role in improving the quality of ECG signals. The raw ECG signals obtained from the dataset contained noise such as baseline wander and high-frequency interference.

After applying bandpass filtering, the signal became smoother and the QRS complexes became more prominent. This improved signal quality ensured accurate detection of R-peaks and reliable feature extraction.

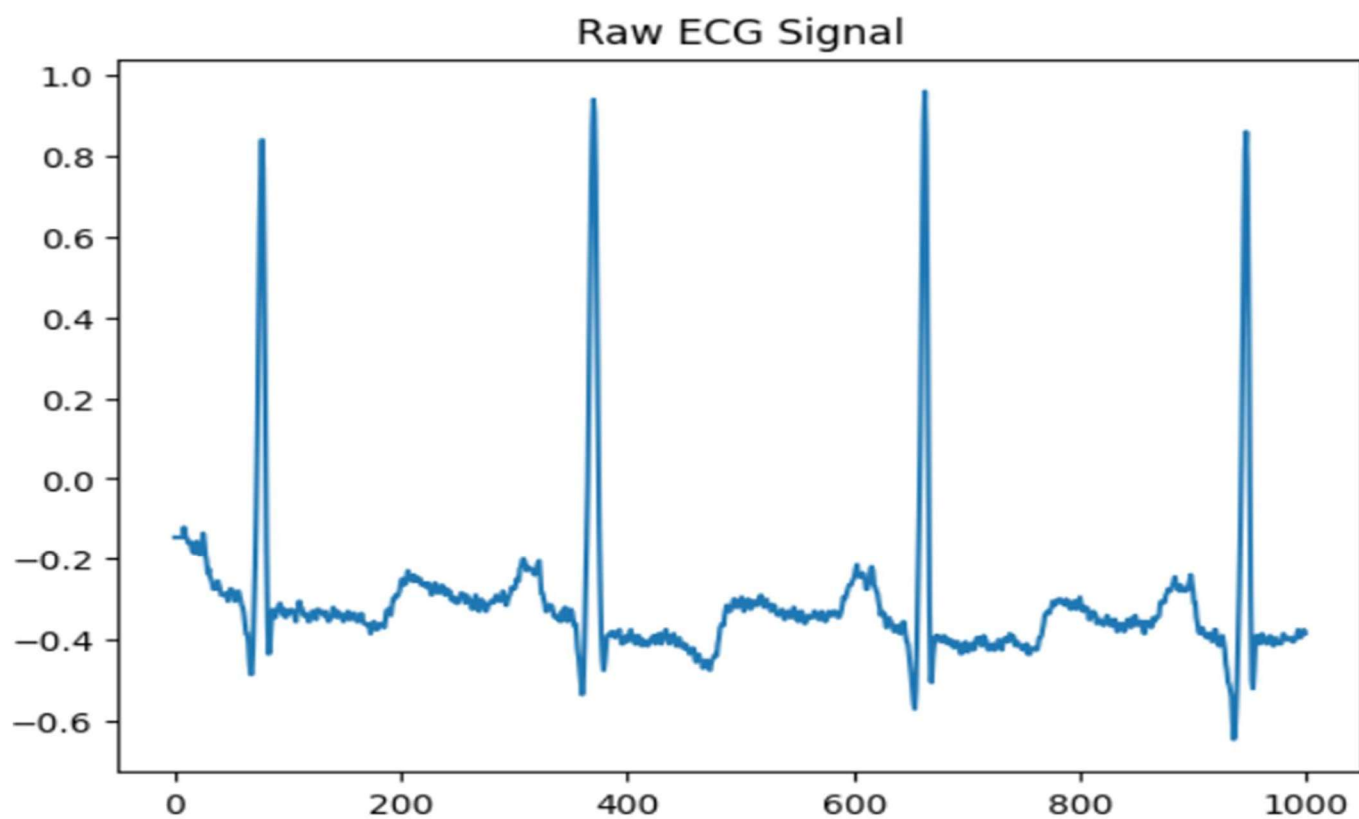


Fig 5.1 RAW ECG SIGNAL

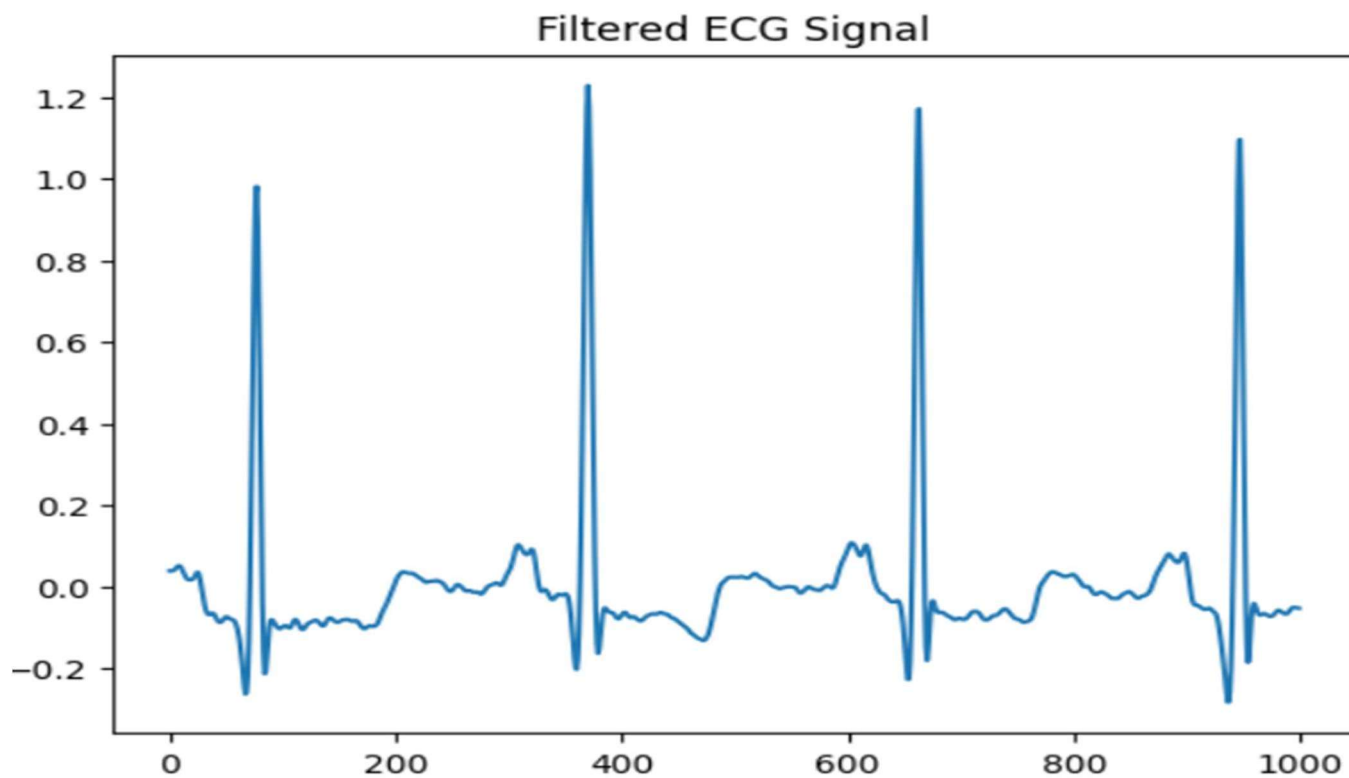


Fig 5.2 FILTERED ECG SIGNAL

5.3 R – PEAK DETECTION RESULTS

R-peak detection was successfully performed using peak detection algorithms. The detected peaks correspond to the QRS complexes of the ECG signal.

The results show that the algorithm accurately identifies the peaks, which are essential for calculating RR intervals and analyzing heart rhythm. Accurate R-peak detection significantly improves the overall performance of the system.

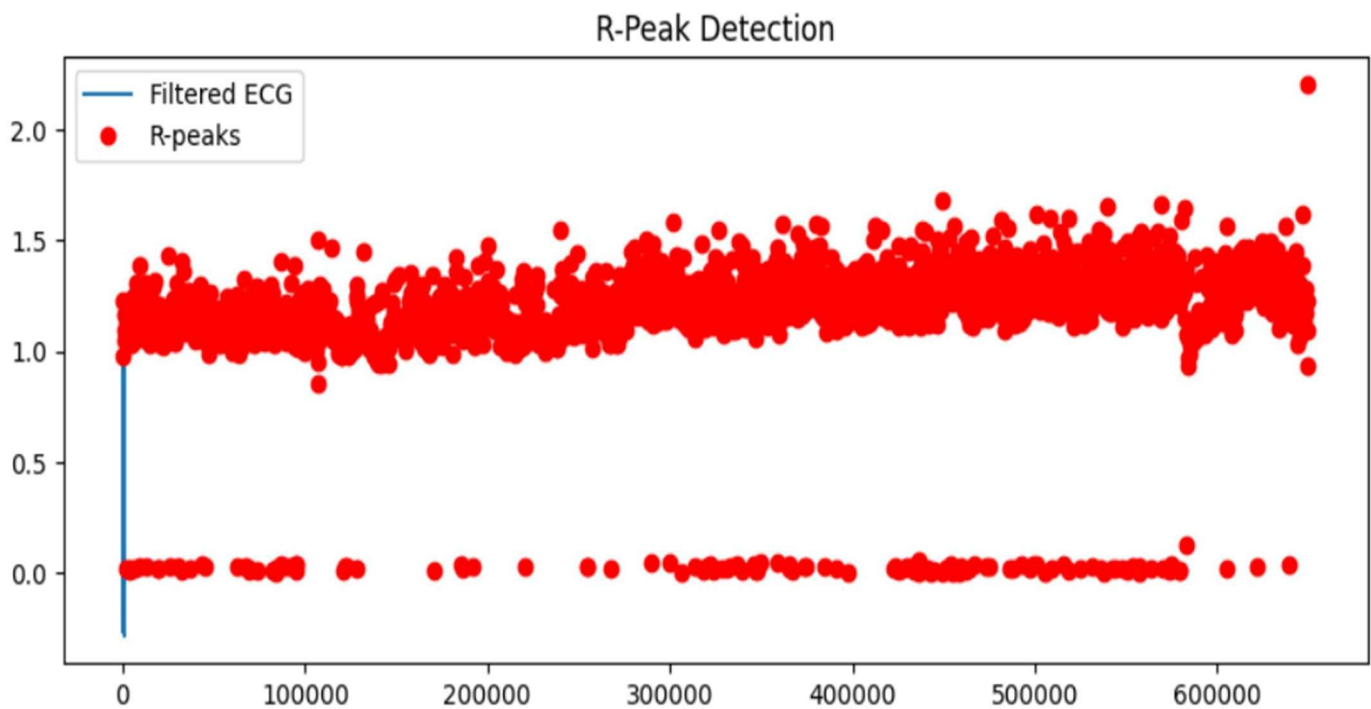


Fig 5.3 R – PEAK DETECTION

5.4 FEATURE EXTRACTION RESULTS

Feature extraction was performed on segmented ECG signals. Various statistical features such as mean, standard deviation, maximum value, minimum value, median, and range were extracted.

These features effectively represent the characteristics of ECG signals and help distinguish between normal and abnormal heartbeats. The extracted feature set was used as input for machine learning models.

Check Data

```
print("Total Segments:", len(all_segments))
print("Total Labels:", len(all_labels))
```

Total Segments: 13418
Total Labels: 13418

Feature Extraction

```
def extract_features(segment):
    return [
        np.mean(segment),
        np.std(segment),
        np.max(segment),
        np.min(segment),
        np.median(segment),
        np.ptp(segment)
    ]
X = [extract_features(seg) for seg in all_segments]
y = all_labels
print("Sample Feature:", X[0])
```

... Sample Feature: [np.float64(0.00908912581799442), np.float64(0.2189319519317806), np.float64(1.227471158947131), np.float64(-0.20248250391944458), np.float64(-0.0534821112674

Check Data

```
print("Total Segments:", len(all_segments))
print("Total Labels:", len(all_labels))
```

Total Segments: 13418
Total Labels: 13418

Feature Extraction

```
def extract_features(segment):
    return [
        np.mean(segment),
        np.std(segment),
        np.max(segment),
        np.min(segment),
        np.median(segment),
        np.ptp(segment)
    ]
X = [extract_features(seg) for seg in all_segments]
y = all_labels
print("Sample Feature:", X[0])
```

... 2581799442), np.float64(0.2189319519317806), np.float64(1.227471158947131), np.float64(-0.20248250391944458), np.float64(-0.05348211126742679), np.float64(1.4299536628665757)]

Fig 5.4 FEATURE EXTRACTION

5.5 MODEL PERFORMANCE EVALUATION

The performance of the system was evaluated using two machine learning models: Support Vector Machine (SVM) and Random Forest.

The dataset was divided into training and testing sets using an 80–20 split. The models were trained using extracted features and tested on unseen data.

Train-Test Split

```
▶ x_train, x_test, y_train, y_test = train_test_split(  
    x, y, test_size=0.2, random_state=42  
)  
print("Train size:", len(x_train))  
print("Test size:", len(x_test))
```

```
... Train size: 10734  
    Test size: 2684
```

Fig 5.5 TRAIN SIZE AND TEST SIZE

Accuracy

```
▶ print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))  
print("RF Accuracy:", accuracy_score(y_test, y_pred_rf))  
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))
```

```
... SVM Accuracy: 0.9631147540983607  
    RF Accuracy: 0.9713114754098361  
    Confusion Matrix:  
      [[1810   10]  
       [  67  797]]
```

Fig 5.6 SUPPORT VECTOR MACHINE AND RANDOM FOREST ACCURACY

The results indicate that Random Forest performs slightly better due to its ability to handle complex data and reduce overfitting.

5.6 CONFUSION MATRIX ANALYSIS

The confusion matrix provides a detailed analysis of classification performance. It shows the number of correctly and incorrectly classified samples.

The matrix includes the following components:

- True Positive (TP): Correctly classified abnormal signals
- True Negative (TN): Correctly classified normal signals
- False Positive (FP): Normal signals classified as abnormal
- False Negative (FN): Abnormal signals classified as normal

The results show that the number of correct predictions is significantly higher than incorrect ones, indicating good model performance.

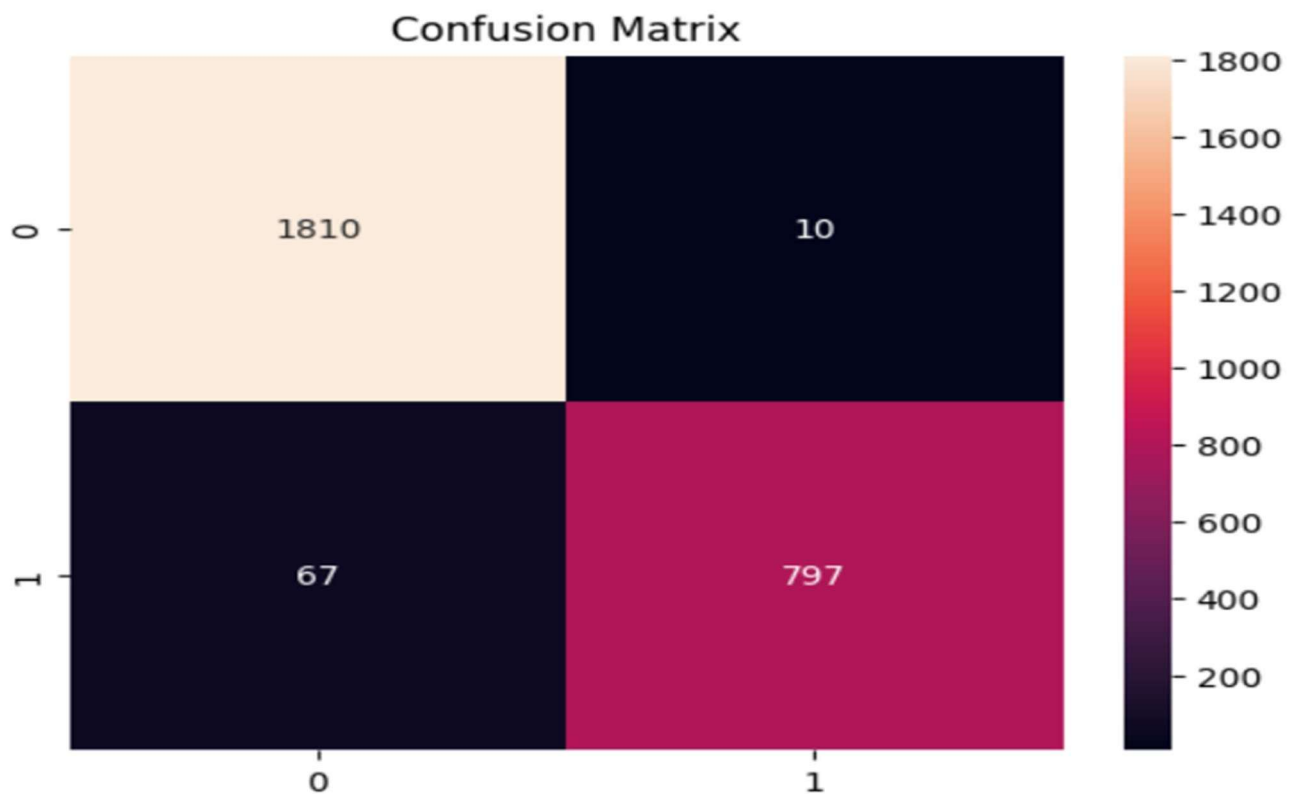


Fig 5.7 CONFUSION MATRIX ANALYSIS

5.7 CLASSIFICATION REPORT ANALYSIS

The classification report includes important performance metrics such as precision, recall, and F1-score.

- Precision indicates the accuracy of positive predictions
- Recall measures the ability of the model to detect actual positives
- F1-score provides a balance between precision and recall

The values obtained demonstrate that the model performs effectively in classifying ECG signals.

Classification Report

 <code>print(classification_report(y_test, y_pred_rf))</code>					
...		precision	recall	f1-score	support
	0	0.96	0.99	0.98	1820
	1	0.99	0.92	0.95	864
	accuracy			0.97	2684
	macro avg	0.98	0.96	0.97	2684
	weighted avg	0.97	0.97	0.97	2684

Fig 5.8 CLASSIFICATION REPORT ANALYSIS

5.8 COMPARATIVE ANALYSIS

A comparison between SVM and Random Forest models was performed.

Random Forest showed better stability and higher accuracy, while SVM provided good classification boundaries.

Overall, Random Forest is more suitable for this application due to its robustness and ability to handle noisy data.

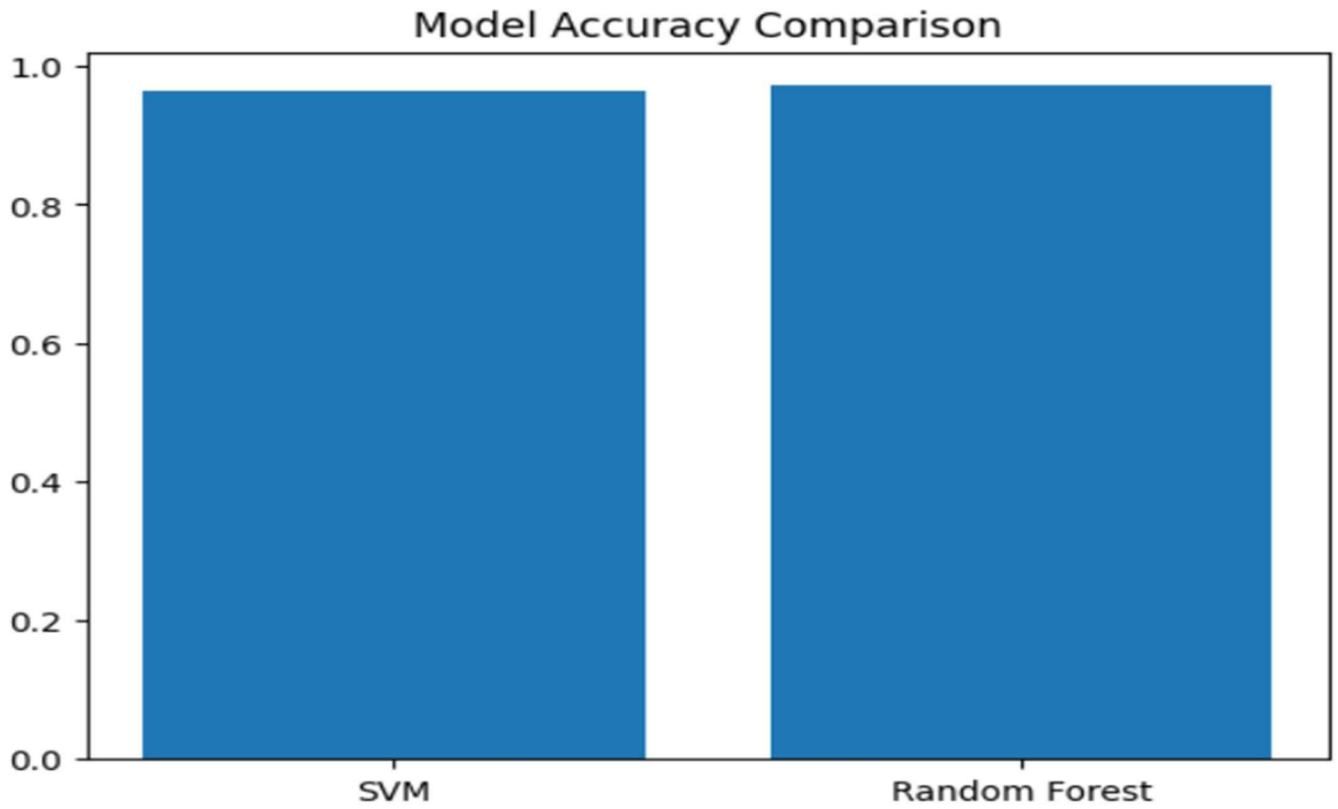


Fig 5.9 MODEL ACCURACY COMPARISON (SVM VS RF)

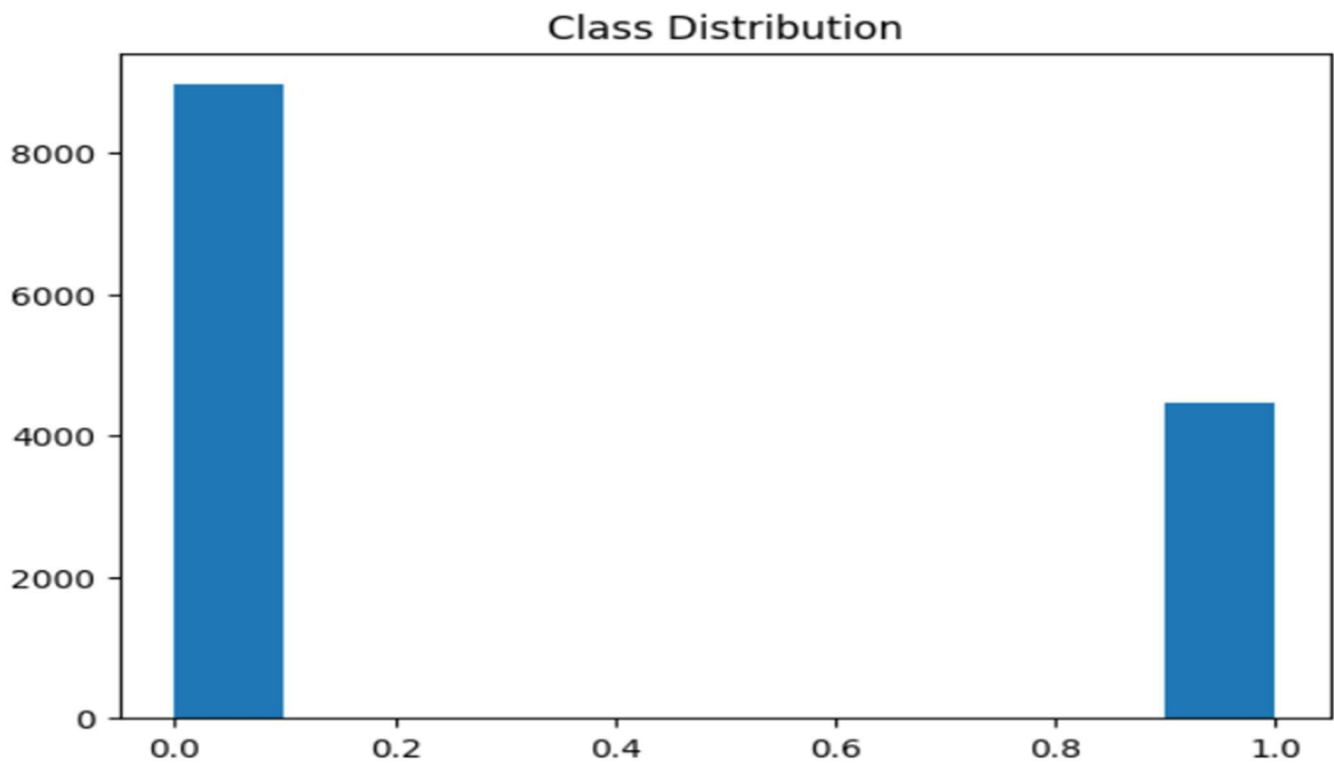


Fig 5.10 CLASS DISTRIBUTION

5.9 OBSERVATIONS

The following observations were made during the implementation:

- Preprocessing significantly improves signal quality
- Accurate R-peak detection is essential for reliable feature extraction
- Feature selection improves classification performance
- Random Forest performs better than SVM in most cases

5.10 SUMMARY

This section presented the results and discussion of the proposed ECG signal processing system. The results demonstrate that the system can effectively classify ECG signals into normal and abnormal categories using machine learning techniques.

The next section presents the conclusion and future scope of the project.

CONCLUSION

In this project, an automated system for ECG signal processing and arrhythmia detection using machine learning techniques was successfully developed. The system utilized ECG data from the MIT-BIH Arrhythmia Database, which provided annotated signals for analysis.

The raw ECG signals were first preprocessed using bandpass filtering to remove noise such as baseline wander and high-frequency interference. This improved the quality of the signal and made it suitable for further analysis. R-peak detection was then performed to identify the QRS complexes, which are essential for analyzing heart rhythm.

After segmentation, important features such as mean, standard deviation, maximum value, minimum value, median, and range were extracted from the ECG signals. These features effectively represented the characteristics of the signal and were used as input for machine learning models.

Two machine learning algorithms, Support Vector Machine (SVM) and Random Forest, were implemented and evaluated. The results showed that both models were able to classify ECG signals into normal and abnormal categories with good accuracy. Among the two, Random Forest demonstrated better performance due to its robustness and ability to handle noisy data.

Overall, the project successfully demonstrated how digital signal processing and machine learning can be combined to develop an efficient and automated system for ECG analysis. The system reduces dependency on manual interpretation and provides a reliable approach for early detection of cardiac abnormalities.

APPLICATIONS

The proposed system has several practical applications in the field of healthcare and biomedical engineering:

- It can be used in hospitals and diagnostic centers for automated ECG analysis.
- It helps cardiologists in detecting arrhythmias quickly and accurately.
- It can be integrated with wearable devices such as smartwatches for continuous heart monitoring.
- It supports telemedicine systems by enabling remote diagnosis of heart conditions.
- It can be used in research and development of AI-based healthcare solutions.

These applications highlight the usefulness of the system in improving healthcare efficiency and accessibility.

FUTURE SCOPE

Although the system performs effectively, several improvements can be made in the future to enhance its performance and capabilities:

- Implementation of deep learning models such as Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) for improved accuracy.
- Extension of the system for multi-class classification of different types of arrhythmias.
- Development of a real-time ECG monitoring system using IoT devices.
- Integration with mobile applications for continuous health tracking.
- Use of larger and more diverse datasets to improve generalization.
- Optimization of algorithms for deployment on embedded systems and wearable devices.

These improvements can make the system more advanced, accurate, and suitable for real-world applications.

REFERENCES

- [1] J. Pan and W. J. Tompkins, “A Real-Time QRS Detection Algorithm,” *IEEE Transactions on Biomedical Engineering*, vol. 32, no. 3, pp. 230–236, 1985.
- [2] G. B. Moody and R. G. Mark, “The Impact of the MIT-BIH Arrhythmia Database,” *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001.
- [3] P. de Chazal, M. O’Dwyer, and R. B. Reilly, “Automatic Classification of Heartbeats Using ECG Morphology,” *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 7, pp. 1196–1206, 2004.
- [4] S. Osowski, L. T. Hoai, and T. Markiewicz, “Support Vector Machine-Based Expert System for Reliable Heartbeat Recognition,” *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 4, pp. 582–589, 2004.
- [5] U. R. Acharya et al., “A Deep Convolutional Neural Network Model to Classify Heartbeats,” *Information Sciences*, vol. 415–416, pp. 190–198, 2017.
- [6] Ö. Yıldırım, U. B. Baloglu, and U. R. Acharya, “Arrhythmia Detection Using Deep Learning,” *Computers in Biology and Medicine*, vol. 109, pp. 247–258, 2019.
- [7] A. Sun, “CNN-LSTM with Attention Mechanism for ECG Classification,” *Sensors*, vol. 24, no. 19, 6306, 2024.

APPENDIX A – SOURCE CODE

A.1 IMPORTING LIBRARIES

```
import wfdb
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.signal import butter, filtfilt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler
```

A.2 BANDPASS FILTER FUNCTION

```
def bandpass_filter(sig, fs=360):
    b, a = butter(3, [0.5, 40], btype='bandpass', fs=fs)
    return filtfilt(b, a, sig)
```

A.3 ECG SIGNAL LOADING

```
record = wfdb.rdrecord('100', pn_dir='mitdb')
signal = record.p_signal[:,0]
```

A.4 SIGNAL PREPROCESSING

```
filtered = bandpass_filter(signal)
```

A.5 R – PEAK DETECTION

```
from scipy.signal import find_peaks
peaks, _ = find_peaks(filtered, distance=150)
```

A.6 SEGMENTATION AND LABELING

```
records = ['100','101','102','103','104','105']
all_segments = []
all_labels = []
for rec in records:
    record = wfdb.rdrecord(rec, pn_dir='mitdb')
    annotation = wfdb.rdann(rec, 'atr', pn_dir='mitdb')
    signal = record.p_signal[:,0]
    signal = bandpass_filter(signal)
    peaks = annotation.sample
    symbols = annotation.symbol
    for i in range(1, len(peaks)-1):
        peak = peaks[i]
        if peak-100 >= 0 and peak+100 < len(signal):
            segment = signal[peak-100:peak+100]
            if len(segment) == 200:
                all_segments.append(segment)
                if symbols[i] == 'N':
                    all_labels.append(0)
                else:
                    all_labels.append(1)
```

A.7 FEATURE EXTRACTION

```
def extract_features(segment):
```

```
    return [  
        np.mean(segment),  
        np.std(segment),  
        np.max(segment),  
        np.min(segment),  
        np.median(segment),  
        np.ptp(segment)  
    ]
```

```
X = [extract_features(seg) for seg in all_segments]  
y = all_labels
```

A.8 FEATURE SCALING

```
scaler = StandardScaler()  
X = scaler.fit_transform(X)
```

A.9 TRAIN – TEST SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

A.10 MODEL TRAINING

```
svm_model = SVC()  
svm_model.fit(X_train, y_train)  
rf_model = RandomForestClassifier()  
rf_model.fit(X_train, y_train)
```

A.11 PREDICTION AND EVALUATION

```
y_pred_rf = rf_model.predict(X_test)  
print("Accuracy:", accuracy_score(y_test, y_pred_rf))  
print(confusion_matrix(y_test, y_pred_rf))  
print(classification_report(y_test, y_pred_rf))
```